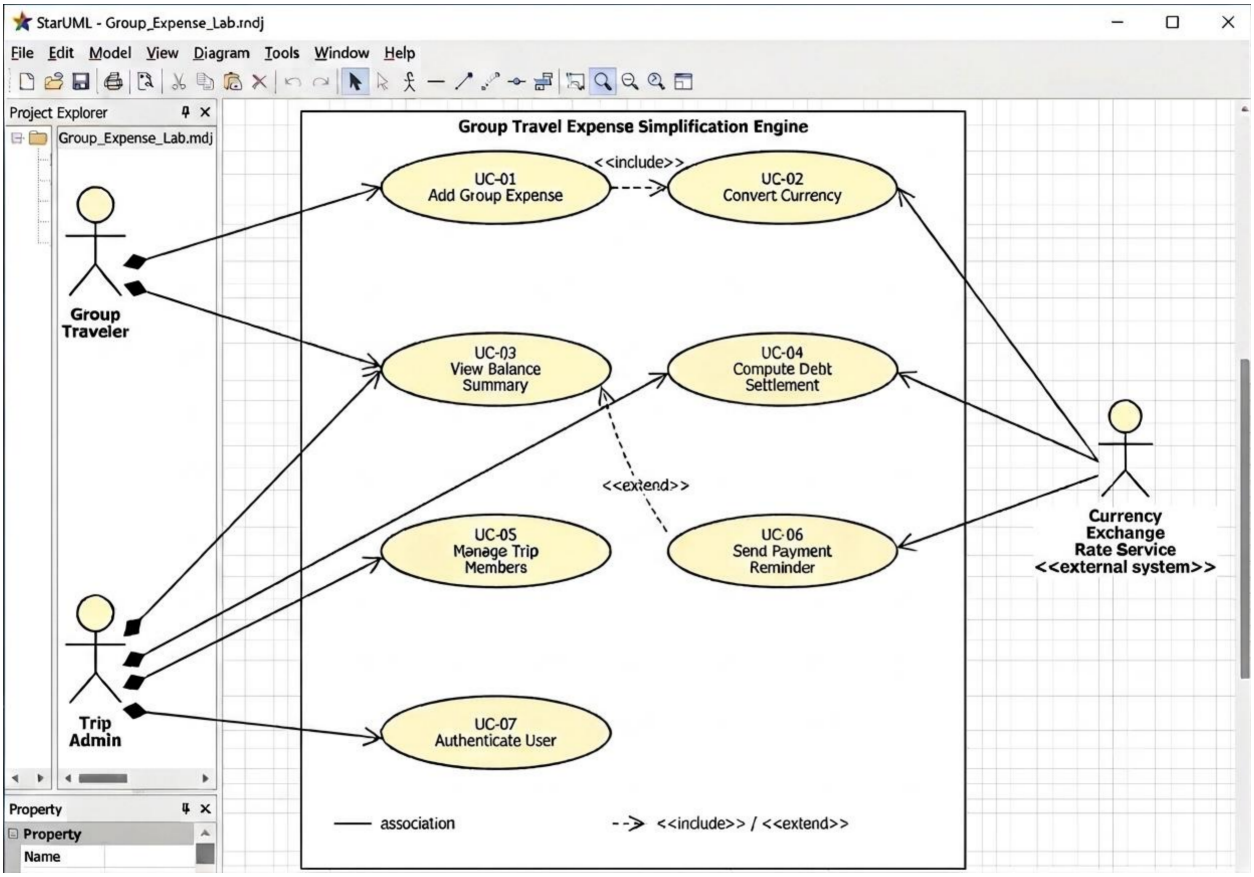
Group Travel Expense Simplification Engine

Problem Statement #37 — UML & Requirements Documentation

Shivansh Sharma
SRN: PES2UG24CS474
SE Lab-1

1. Use Case Diagram



2. Use-Case Flow Specification

Problem Statement #37 — Group Travel Expense Simplification Engine

Use Case ID	UC-04
Use Case Name	Compute Debt Settlement
Primary Actor	Trip Admin
Secondary Actors	Group Traveler (views result), Currency Exchange Rate Service (via included Convert Currency use case)
Preconditions	Trip Admin is logged in (UC-07), trip has at least 1 expense, and expenses are already converted to base currency (UC-02).
Postconditions	Success: settlement saved, shows who pays who + how much, nets to zero. Failure: nothing saved, old balances unchanged.

Main Success Scenario

1. Trip Admin taps “Settle Up” on the trip dashboard.
2. System pulls every expense for the trip (already in base currency).
3. For each traveler, adds up what they paid vs what they owe based on splits.
4. Works out each person's net balance (paid minus owed).
5. Runs the min-flow algorithm on those balances to get smallest set of payments that clears everyone.
6. Checks total debts = total credits (net zero).
7. Shows the settlement to Trip Admin as a list like “A pays B: amount.”
8. Trip Admin confirms.
9. System notifies each traveler of their part and marks trip as Settled.
10. Done.

Alternate Flow

6a. Balances don't add up

6a1. Net-zero check in step 6 fails, usually an expense wasn't converted right.

6a2. System stops, shows error instead of a broken settlement.

6a3. Flags which expense(s) look off, sends admin back to fix conversion (UC-02).

6a4. Admin fixes it, retries Settle Up, goes back to step 2.

6a5. Still broken after 2 tries, use case fails and gets flagged for support.

3. Requirements Table

Problem Statement #37 — Group Travel Expense Simplification Engine

Actors: Group Traveler, Trip Admin, Currency Exchange Rate Service. *(Italic rows = given in the handout, rest written by me.)*

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	*Functional*	*The system shall compute the optimal debt settlement graph using min-flow algorithms to minimize total peer-to-peer payment transactions.*	*High*	*Group balance settles to net zero with minimal payment edges. Fails if sum of debts != sum of credits.*	*Point of the whole app tbh, fewer transfers = less hassle when trip ends.*
FR-002	Functional	The system shall let a Group Traveler log a shared expense (who paid, amount, currency) and split it unevenly across selected participants using percentage, exact amount, or shares.	High	Shares should add up to total when saved (small rounding ok, ~0.01). Don't let it save otherwise.	This is what makes the app different from just splitting evenly. If split logic breaks here everything after (balances, settlement) breaks too.
FR-003	Functional	The system shall convert expenses entered in a foreign currency to the trip's base currency using that day's exchange rate.	High	Foreign currency expense stores original amount + converted amount, using rate from that date, not an old cached one.	Was in the problem statement so had to include it. Needed anyway so amounts across currencies can be compared.
FR-004	Functional	The system shall let the Trip Admin add or remove travelers from a trip. Removed people shouldn't be part of new expense splits but old expenses with them stay in history.	Medium	Remove a traveler -> they don't show up as an option for new expenses, but past expenses/history involving them still show.	People leave/join trips halfway sometimes so didn't want removing someone to mess up past data.
FR-005	Functional	The system shall let a Group Traveler see a live balance summary of what they owe / are owed by each person, in base currency.	High	Updates within a couple seconds of new expense, all balances in the group should sum to 0.	Didn't want people to have to wait till the end of the trip to know where they stand money-wise.
NFR-001	*Nonfunctional*	*The settlement calculation engine must execute in under 100 ms for groups of up to 50 travelers with 500 expenses.*	*High*	*Benchmarking tests confirm target latency and security standards under simulated peak load.*	*given in handout*
NFR-002	Nonfunctional	The system shall encrypt financial/personal data in transit (TLS 1.2+) and at rest (AES-256). No storing raw payment info.	High	Security check confirms traffic is TLS 1.2+ and DB fields are actually encrypted, not plaintext.	Money data + multiple people involved = has to be secure, not really optional for this kind of app.